Decision Tree Classifier
Kaggle - Forest Cover Type Prediction

1. Introduction

Decision trees are widely used in machine learning due to their high interpretability and ability to model complex and nonlinear data relationships. This project develops a CART-based decision tree classifier in Python to forecast forest cover types based on the Forest Cover Type dataset.

The use of CART algorithm for this task is appropriate because CART supports multi-class classification with diverse input features (continuous and binary/categorical), not requiring any form of scaling or normalization, and generates interpretable decision rules. These characteristics make this algorithm a perfect fit for the Forest Cover Type dataset, which includes both numeric features and binary indicator variables.

This work is devoted to the development of a tree using the optimization procedure. The process involves selection of split points by minimizing Gini impurity through a greedy algorithm over the space of feature-threshold pairs. Apart from implementing the algorithm from scratch, the work will involve assessment of its performance on real data as well as verification of its accuracy against scikit-learn's efficient implementation of the decision tree classifier.

2. Machine Learning Method: CART Decision Trees

2.1 Overview of the CART Algorithm

The Classification and Regression Tree (CART) algorithm creates a binary decision tree using a recursive method, whereby the training data set is recursively split into progressively homogenous subgroups. At each step, a splitting criterion that uses one specific attribute with a certain threshold is applied to divide the dataset into two groups, aiming to decrease class homogeneity. The recursion stops when pre-specified conditions are met, such as achieving a maximum tree depth or pure nodes.

2.2 Data Structures

The CART algorithm is implemented using a node structure which holds the decision rule along with the resulting data partition. This setup facilitates easy recursive construction of the tree and efficiently follows the path for predictions, reflecting the hierarchical nature of the model.

Attribute	Type	Meaning
feature_index	int	Index of the feature used to split
threshold	float	Threshold value for the split
left / right	node	References to the left and right child nodes
is_leaf	bool	True when this node makes a prediction
predicted_class	int	Majority class label at each leaf
gini	float	Gini impurity at this node
n_samples	int	Number of training samples that reached this node

2.3 Gini Impurity

Gini impurity is employed in CART to evaluate the quality of each possible split, defined as $1 - \sum_k p_k^2$, where p_k is the proportion of samples belonging to class k. During each iteration, the algorithm chooses the feature and threshold that yield the lowest weighted Gini impurity of the

resulting child nodes. A node is considered pure when all observations fall into the same class, giving a Gini value of zero, while higher values indicate greater class diversity.

Gini impurity is chosen due to its computation efficiency, as it is not dependent on logarithmic calculations such as entropy. Therefore, it works particularly well in situations where a large number of possible splits should be evaluated.

2.4 Training and Prediction Process

The CART algorithm is trained through a recursive process that splits the dataset into progressively more homogeneous subsets. Starting from the root node, it evaluates different feature-threshold combinations (j, θ) to identify the split that minimizes the weighted Gini impurity of the resulting child nodes.

After selecting the best split, the data is divided into two groups of left and right nodes, and the same process is applied recursively on both. This terminates until certain criteria, such as reaching a maximum depth, having too few samples in a node, or when the node becomes pure. This results in a hierarchical structure that can capture complex patterns in the data.

Predictions occur when the new sample passes from the root to a leaf node, following the learned decision rules at each split: routed to the left child node when $x_j \leq \theta$, and to the right otherwise. The final prediction is determined by the majority class of the samples in the leaf node.

2.5 Feature Importance

Feature importance is computed based on the total reduction in Gini impurity contributed by each feature across all splits in the tree. Larger reductions indicate higher feature importance. For a split node t using feature j , the contribution is calculated as:

$$impor\tan c\, e_j += \frac{n_t}{n_{total}}\left[Gini(t) - \frac{n_L}{n_t}Gini(L) - \frac{n_R}{n_t}Gini(R)\right]$$

, where L and R are the left and right child nodes. These contributions are then summed across all splits where the feature appears and normalized to produce the final importance scores, giving an intuitive measure of how much each feature contributes to the model's performance.

3. Optimization Problem Formulation

3.1 The Local Optimization at Each Node

At each node, CART solves a local minimization problem for obtaining the best split. Specifically, it finds the feature j and the threshold θ that minimize the weighted Gini impurity of the subsequent child nodes: $\min_{j,\theta}\left(\frac{n_L}{n_t}Gini(S_L) + \frac{n_R}{n_t}Gini(S_R)\right)$. Such an optimization criterion encourages splits that produce more homogeneous subsets, thereby improving the classification accuracy of the tree.

3.2 Problem Classification

Decision tree training constitutes a non-convex combinatorial optimization problem. Identifying the globally optimal tree would require evaluating all possible feature–threshold combinations across all levels in parallel, which is known to be NP-hard.

CART solves this problem through a greedy method that separates it into a series of local optimization steps. The algorithm selects the best split based on the impurity criterion at each individual node level, without backtracking.

The search space is discrete because candidate thresholds are usually chosen from midpoints between consecutive unique feature values. This drastically reduces the number of possible splits while preserving all meaningful partitions, making the full tree computationally tractable.

3.3 Equivalent Formulation

The minimization of the weighted Gini impurity of the child nodes is equivalent to the maximization of the reduction in impurity, which is referred to as Gini gain:

$$\Delta Gini = Gini(S_t) - \left[\left(\frac{n_L}{n_t}\right) Gini(S_L) + \left(\frac{n_R}{n_t}\right) Gini(S_R)\right]$$

In the CART method, splits are selected to maximize the Gini gain, making the subsets homogeneous. Mathematically, both perspectives are identical, but interpreting the impurity reduction gives a more intuitive understanding of how each split improves the model.

An alternative criterion is information gain (entropy reduction), defined as $H(S) = -\sum_k p_k \log p_k$. In practice, both criteria produce similar trees.

3.4 Stopping Criteria as Constraints

There are multiple criteria for the tree growing process to stop, which can be regarded as constraints on the optimization problem. These conditions limit further splitting unless it brings any useful improvements to the tree, striking a balance between model complexity and generalization while reducing the risk of overfitting.

Constraint	Effect
$\boldsymbol{depth(t) \geq d_{max}}$	Limits tree complexity; controls overfitting
$\boldsymbol{n_t < n_{min-samples-split}}$	Prevents splits on very small nodes
$\boldsymbol{n_L < n_{min-samples-leaf}}$ or $\boldsymbol{n_R < n_{min-samples-leaf}}$	Ensures minimum child size
$\boldsymbol{\lvert C_t \rvert = 1}$ (pure node)	Natural termination; no gain possible
$\boldsymbol{\Delta Gini = 0}$	No informative split exists

3.5 Prediction as Tree Traversal

During inference, every sample travels from the root to the leaf nodes using the split rules learned during training. The output prediction is based on the majority class among all observations at the relevant leaf node.

Prediction is quite different from training in the sense that it doesn't require solving an optimization problem, but only comparison operations.

4. Dataset

4.1 Forest Cover Type Dataset

The Forest Cover Type Prediction dataset comprises cartographic measurements for the Roosevelt National Forest in Northern Colorado. The goal is to classify the forest cover type using the topographic maps and geographic information system (GIS) data.

ID column has been removed from the dataset, now containing 15,120 samples and 54 attributes. Out of all attributes, there are 10 continuous topographic variables, as well as binary indicators for 4 wilderness areas and 40 soil types. Without missing values, no imputation is needed.

4.2 Feature Distribution Insights

The distributions of key variables are examined to gain insight into the structure of predictions.

Elevation is the most discriminative feature. Certain classes, such as Krummholz (Type 7), are concentrated at high elevations (above ~3,500m), while others like Cottonwood/Willow (Type 4) occur at low elevations (~2,200 meters). This variable is chosen as the root split of the decision tree due to its clear separation.

Variables such as *Slope* and *Horizontal Distance to Roadways* show considerable overlap among the classes, limiting their potential to be used for splitting at the root. However, they may still provide useful information if paired with other variables at deeper tree levels.

The *Horizontal Distance to Fire Points* is moderately differentiated across the classes, implying a correlation with the vegetation patterns but not a high degree of discriminability alone.

The large overlap found in many features implies that none of them can alone achieve accurate classification. Therefore, there is a need for an algorithm like CART, which allows the combination of several variables to capture complex interactions across features.

4.3 Key EDA Findings

Class balance: Each cover type contains exactly 2,160 samples, making this a perfectly balanced dataset. Accuracy is thus a suitable evaluation metric to use, and class weighting or resampling are not required.

Feature correlations: Some topographic predictors display strong correlations. For instance, Hillshade 9am and 3pm have a significant negative correlation ($r=-0.78$), as expected because of natural sunlight patterns where the slope facing the East would be exposed to morning sun but shaded in the afternoon, and vice versa for west-facing slopes. Another feature pair with significant negative correlation ($r=-0.61$) is Hillshade Noon and Slope, whereas there is a positive correlation ($r=0.65$) between Horizontal and Vertical Distance to Hydrology

Nonlinear class structure: The target variable, *Cover_Type*, exhibits weak linear correlation with individual predictors ($|r|$ near 0), which suggests that the decision boundary is highly nonlinear. Therefore, a tree-based algorithm can be used to capture complex interactions through hierarchical splits.

4.5 Train/Validation/Test Split

The data was stratified based on classes and split into 70% training (10,584 samples), 15% validation (2,268 samples), and 15% test (2,268 samples). Stratification guarantees consistency in terms of class proportions within each subset.

The validation set is only used for hyperparameter tuning (max_depth), while the test set is reserved for final model evaluation without participation in training. This ensures that the model is evaluated in an unbiased and reliable way.

5. Model Implementation

5.1 Implementation Overview

The system architecture is modular and has several components for construction, splitting, prediction, and evaluation. Nodes are used to store the splitting criteria or prediction outcomes. The training process includes finding the optimal feature-threshold combination using Gini impurity and continuing until stopping conditions are met. Prediction uses a traversal method to find the appropriate leaf node, while other utilities are used to measure feature importance and inspect tree structure.

5.2 Training Algorithm

The training process begins with all samples at the root node. The algorithm looks for the best split at each node by looking at all possible pairs of feature and threshold. For each feature, candidate thresholds are selected from the midpoints between consecutive unique values. The dataset is split into left and right subsets for each candidate split, and the weighted Gini impurity is then calculated. The optimal split is the one that minimizes the weighted impurity of the child nodes. Once the best split is identified, the dataset is divided accordingly, and the same procedure is applied recursively to each subset. The recursion continues until one of the stopping criteria is met, such as reaching the maximum depth, having too few samples, or achieving a pure node. This results in a binary tree structure that approximates the underlying data distribution.

5.3 Hyperparameter Tuning

The maximum depth is the most important hyperparameter in the decision tree model because it controls how complex the tree is. A deeper tree can capture more detailed patterns but increases the risk of overfitting. To find the best depth, the validation dataset is used to test a number of different values. The model is trained at different levels of depth, and the accuracy of the validation set is used to judge how well it works. The results show that shallow trees underfit the data, while very deep trees begin to overfit. The optimal depth of 14 was selected and used for all final evaluations. The re-trained custom tree at this depth grew to 1,959 nodes with 980 leaf nodes, representing 980 distinct decision regions in the 54-dimensional feature space.

5.4 Computational Complexity

The split selection process is part of the implementation that takes the most computing power. At each node, the algorithm evaluates all features and possible thresholds, leading to a time complexity of approximately $O(pn \log n)$, where p is the number of features and n is the number of samples.

Compared to optimized implementations such as scikit-learn, the from-scratch version is significantly slower due to the use of Python loops and the absence of low-level optimizations. However, this trade-off allows for a clearer understanding of the underlying algorithm.

6. Experimental Analysis and Results

6.1 Summary Performance

The performance of the custom decision tree is evaluated on training, validation, and test datasets, and compared with scikit-learn's DecisionTreeClassifier using the same hyperparameters. In general, both models get very similar results on all metrics. The difference in test accuracy between the custom implementation and the scikit-learn model is small, which means that the algorithm's core logic has been implemented correctly.

The predictive performance is almost the same, but the computational efficiency is very different. The custom implementation takes a lot more time to train because it doesn't have the low-level optimizations that scikit-learn's implementation does.

Metric	Custom Tree	Sklearn Tree	Delta
Training Accuracy	0.9434	0.9434	0.00000
Validation Accuracy	0.7888	0.7897	0.00088
Test Accuracy	0.7954	0.7953	0.00088
Max Depth	14	14	-
Total Nodes	1,959	1,951	8
Leaf Nodes	980	976	4
Training Time	61.8s	0.13s	~475x

6.2 Per-Class Performance

Performance is very different depending on the type of forest cover. Some classes are consistently predicted with high precision and recall, while others are more difficult to classify. Classes associated with more distinct environmental conditions tend to achieve higher F1-scores. For example, forest types that occur at extreme elevations are easier to separate, as their feature distributions are clearly different from other classes. On the other hand, some mid-elevation classes have lower recall and F1-scores. These classes share some of the same areas in the feature space, especially when it comes to things like elevation and slope. Because of this, the model has a hard time making clear decisions in these cases.

6.3 Confusion Matrix Analysis

The confusion matrix helps us understand more about how classification errors are spread out. Most of the predictions are on the diagonal, which means that the model works well overall. But the most common mistakes happen when classes have similar environmental traits. Some mid-range forest types are often mixed up with each other, which is similar to what happens in the feature space. On the other hand, classes with more distinct feature profiles have very few mistakes in classification. This means that the decision tree works well when class boundaries are not very clear, but it doesn't work as well when they are.‹

6.4 Feature Importance Analysis

Feature importance analysis shows that certain variables play a dominant role in the model's decisions. Among all features, elevation consistently appears as the most important predictor. This is expected, as elevation is strongly linked to ecological conditions that determine forest cover type. The model uses this feature early in the tree to create major splits that separate broad groups of classes. Other features, such as distance to hydrology and roadways, also contribute to the model, but their impact is more moderate. Binary soil type indicators individually have limited influence but collectively help refine classification decisions at deeper levels of the tree.

6.5 Overfitting Analysis

A comparison between training and test performance reveals a noticeable gap, indicating moderate overfitting. While the model achieves high accuracy on the training data, its performance decreases unseen data. This behavior is expected for decision trees, especially when the tree grows deep. The model tends to capture fine-grained patterns in the training data that do not generalize well. This highlights an inherent limitation of single decision trees. Although they are flexible and interpretable, they can suffer from high variance. Techniques such as pruning or ensemble methods could be used to improve generalization performance.

6.6 Tree Structure Insight

To better understand how the model makes decisions, a shallow version of the tree (e.g., max depth = 3 or 4) is visualized. The visualization shows that the root node splits on elevation, confirming that this is the most important feature in separating forest cover types. Subsequent splits involve features such as hillshade and distance-related variables, which help refine classification within each elevation range. This observation is consistent with the feature importance analysis and highlights how the model builds hierarchical decision rules based on the most informative variables.

7. Discussion and Conclusion

7.1 Key Findings

Overall, the results confirm that the custom implementation of the CART decision tree is correct and behaves consistently with the standard scikit-learn model. The difference in test accuracy between the two models is very small, which suggests that the splitting logic and recursive structure have been implemented properly.

In terms of performance, achieving 79.5% accuracy on a 7-class classification problem is a solid result for a single decision tree, especially without using any ensemble techniques or feature engineering. One of the most important observations is the role of elevation. It consistently appears as the most influential feature and is used at the top levels of the tree to separate major groups of forest types. This aligns well with domain intuition, since different vegetation types are strongly associated with altitude.

At the same time, the model struggles with certain classes, particularly those located in mid-elevation ranges. These classes tend to overlap in multiple features, making it harder for the model to distinguish between them. This explains why their precision and recall are lower compared to other classes.

Another key takeaway is the trade-off between accuracy and efficiency. While the custom implementation achieves comparable predictive performance, it is significantly slower than the scikit-learn version. This highlights the importance of optimized implementations in real-world applications.

7.2 Limitations and Future Work

Despite its strengths, the decision tree model has several limitations.

Single decision tree overfitting: The gap between training and test accuracy (~14%) highlights the high variance of an unpruned decision tree. While the model performs well on the training data (0.839), it captures some noise and does not generalize equally well to unseen data. This limitation is common for single decision trees, especially when the tree grows deep. Techniques such as cost-complexity pruning or setting a minimum number of samples per leaf could help mitigate the issue by controlling model complexity, although this may slightly lower training accuracy.

Ensemble models: Performance can be improved by using ensemble models like Random Forests, which combine the predictions of multiple decision trees trained on random subsets of the data. This reduces variance and leads to better generalization than a single decision tree.

Model bias and flexibility: While decision trees are easy to interpret, their piecewise-constant structure may result in inefficiencies if used to model smoothly changing or complex decision boundaries. Therefore, this can lead to weaker performance when class boundaries are not well aligned with axis-aligned splits.

Computational efficiency: The current version relies on standard Python operations, which makes it relatively slow. Using vectorized operations, or tools such as Numba or Cython, could significantly speed up the training process.

7.3 Conclusion

In this project, a CART decision tree classifier was successfully implemented from scratch and applied to a multi-class classification problem. The results demonstrate that the custom model achieves performance comparable to a standard library implementation, confirming both correctness and effectiveness.

Beyond performance, the project provides a deeper understanding of how decision trees work as an optimization problem. The process of selecting splits based on impurity reduction and building the tree recursively highlights the underlying structure of the algorithm.

Overall, the decision tree proves to be a powerful and interpretable model, but it also has clear limitations in terms of overfitting and sensitivity to overlapping classes. These findings suggest that while decision trees are a strong baseline, more advanced methods can be explored to further improve performance.